Daniel Henrique Charlo

Murilo Della Coletta

Otávio Saturnino

SGMP

Limeira-SP

2026

Sumário

1 Introdução....4
2 Levantamento de Requisitos....5
2.1 Requisitos Funcionais....5
2.2 Requisitos Não Funcionais....6
2.3 MoSCoW....6
3 Metodologia Ágleis (KANBAN)....8
4 Prototipagem (Figma)....9
5 Diagrama de Classes e Banco de Dados....10
6 Diagrama de Sequência....11

PROPOSTA

Gestão de Manutenção Predial

Situação Problema: "O Senai enfrenta situações diárias que envolve lidar com centenas de solicitações mensais, desde lâmpadas queimadas até problemas hidráulicos complexos. A falta de transparência e a demora no atendimento são as principais reclamações de funcionários e alunos."
"O Senai precisa de um aplicativo para controle de toda a estrutura predial, mas precisa primeiro de uma infraestrutura robusta no servidor. A ideia é oferecer transparência total: um chamado é aberto e acompanha cada etapa até a resolução."

Desafio: "Sua equipe será responsável pelo Back-End do 'PredialFix', uma plataforma de chamados de manutenção predial. O foco é criar uma API segura que conecte funcionários, organizando o fluxo de trabalho e priorizando urgências."

Funcionalidades Essenciais:
1. **Gestão de Usuários (Multi-nível):** Autenticação diferenciada para Usuários (abrem chamados) e Responsáveis (atualizam chamados).
2. Abertura de Chamados: Funcionalidade para registrar problemas (Tipo: Elétrica/Hidráulica/Outros, Descrição, Local).
3. Workflow de Atendimento: Lógica para atualizar o status do chamado (Aberto -> Em Análise -> Em Execução -> Concluído).
4. Histórico da Unidade: Consulta de todos os serviços já realizados em um determinado espaço ou área comum.
5. Notificações de Progresso: Feedback simulado sobre a evolução do chamado (ex: "Técnico a caminho", "Serviço finalizado").

Requisitos e Restrições:
- Back-End desenvolvido em Laravel (PHP).
- Banco de dados relacional MySQL.
- Utilizar o Eloquent ORM para facilitar a interação com o banco.
- Validação rigorosa dos dados de entrada (Request Validation).
- API RESTful com respostas em JSON.
- Documentação clara dos endpoints (rotas) da API.
- Entrega via link do repositório GitHub.

1 Introdução

Este documento apresenta a documentação técnica do sistema **PredialFix**, uma API desenvolvida para gerenciar chamados de manutenção predial no SENAI. O sistema tem como objetivo organizar o fluxo de solicitações, melhorar o tempo de atendimento e garantir maior transparência no acompanhamento dos serviços.

Atualmente, a instituição enfrenta dificuldades no controle das demandas, como demora no atendimento e falta de acompanhamento dos chamados. Diante disso, o PredialFix foi proposto para centralizar as informações e permitir que usuários acompanhem cada etapa do processo, desde a abertura até a conclusão do serviço.

O sistema será desenvolvido em Laravel, utilizando arquitetura RESTful, com foco em segurança, organização e escalabilidade. Esta documentação descreve os requisitos, funcionamento e estrutura do back-end, garantindo um desenvolvimento alinhado às necessidades do projeto.

2 Levantamento de Requisitos

O levantamento de requisitos consiste na etapa de identificação, análise e documentação das necessidades que o sistema deve atender. Essa fase é fundamental para garantir que o sistema desenvolvido esteja alinhado com os objetivos e necessidades dos usuários e da organização.

Durante o processo de levantamento foram definidos requisitos funcionais e requisitos não funcionais. Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, ou seja, as ações e serviços que o sistema deve executar, como cadastro de usuários, abertura de ordens de serviço e registro de ativos.

Já os requisitos não funcionais definem as características de qualidade do sistema, como desempenho, segurança, disponibilidade e usabilidade. Esses requisitos estabelecem restrições e padrões que devem ser seguidos durante o desenvolvimento do sistema.

Para auxiliar na priorização das funcionalidades foi aplicada a técnica MoSCoW, que classifica os requisitos em quatro categorias: Must Have, Should Have, Could Have e Won't Have**,** permitindo identificar quais funcionalidades são essenciais para o funcionamento do sistema e quais podem ser implementadas em versões futuras.
Essa etapa é essencial para garantir que o desenvolvimento ocorra de forma estruturada, reduzindo riscos e aumentando a qualidade do produto final.

2.1 Requisitos Funcionais

REQUISITOS FUNCIONAIS
• RF01 – O sistema deve permitir cadastro de usuários. • RF02 – O sistema deve permitir edição e exclusão de usuários. • RF03 – O sistema deve permitir diferentes perfis de acesso (Administrador, Técnico, Solicitante). • RF04 – O sistema deve permitir autenticação por login e senha. • RF05 – O sistema deve permitir recuperação de senha. • RF06 – O sistema deve registrar logs de acesso. • RF07 – O sistema deve permitir cadastro de ativos (equipamentos e estruturas). • RF08 – O sistema deve permitir editar e excluir ativos. • RF09 – O sistema deve registrar localização do ativo. • RF10 – O sistema deve registrar número de patrimônio. • RF11 – O sistema deve manter histórico de manutenção por ativo. • RF12 – O sistema deve permitir abertura de ordem de serviço. • RF13 – O sistema deve permitir classificar a OS (preventiva, corretiva, emergencial). • RF14 – O sistema deve permitir anexar fotos e documentos à OS. • RF15 – O sistema deve permitir definir prioridade da OS. • RF16 – O sistema deve permitir alterar o status da OS (Aberta, Em andamento, Concluída, Cancelada). • RF17 – O sistema deve registrar data, hora e responsável por cada alteração de status. • RF18 – O sistema deve gerar número único para cada OS. • RF19 – O sistema deve manter histórico completo da OS. • RF20 – O sistema deve permitir encerramento da OS com descrição do serviço executado.

- RF21 – O sistema deve permitir cadastro de planos de manutenção preventiva.
- RF22 – O sistema deve permitir definir periodicidade das manutenções.
- RF23 – O sistema deve gerar automaticamente ordens de serviço com base no cronograma.
- RF24 – O sistema deve emitir alertas de manutenção programada.
- RF25 – O sistema deve permitir registrar custos de peças.
- RF26 – O sistema deve permitir registrar custos de mão de obra.
- RF27 – O sistema deve vincular custos a uma ordem de serviço.

2.2 Requisitos Não Funcionais

REQUISITOS NÃO FUNCIONAIS

- RNF01 – O sistema deve possuir controle de acesso baseado em perfil.
- RNF02 – O sistema deve armazenar senhas de forma criptografada.
- RNF03 – O sistema deve suportar no mínimo 100 usuários simultâneos.
- RNF04 – O tempo de resposta das operações deve ser inferior a 3 segundos.
- RNF05 – O sistema deve processar geração de relatórios em até 5 segundos.
- RNF06 – O sistema deve possuir disponibilidade mínima de 99%.
- RNF07 – O sistema deve realizar backup automático diário.
- RNF08 – O sistema deve permitir restauração de dados em caso de falha.
- RNF09 – O sistema deve possuir interface responsiva.
- RNF10 – O sistema deve ter interface intuitiva e de fácil navegação.
- RNF11 – O sistema deve possuir arquitetura escalável.
- RNF12 – O sistema deve permitir atualização sem perda de dados.

2.3 MoSCoW

	Must Have	Should Have	Could Have	Won't Have
Requisitos Funcionais	• RF01 • RF03 • RF04 • RF07 • RF09 • RF10 • RF12 • RF13 • RF15 • RF16 • RF18 • RF20	• RF02 • RF08 • RF11 • RF17 • RF19 • RF21 • RF22	• RF05 • RF06 • RF14 • RF23 • RF24 • RF25 • RF26 • RF27	
Requisitos	• RNF01	• RNF03	• RNF08	

Não Funcionais	• RNF02 • RNF04 • RNF07	• RNF05 • RNF09 • RNF10	• RNF11	

3 Metodologia Ágleis (KANBAN)

Para o desenvolvimento do sistema foi adotada a **metodologia ágil Kanban**, que tem como objetivo organizar e controlar o fluxo de trabalho de forma visual, permitindo maior eficiência na execução das atividades e acompanhamento contínuo das tarefas.

O Kanban é baseado na utilização de um quadro visual, no qual as atividades são distribuídas em colunas que representam as etapas do processo de desenvolvimento, como por exemplo: A Fazer, Em Andamento e Concluído. Cada tarefa é representada por um cartão que se movimenta entre essas colunas conforme o progresso do trabalho.

A utilização dessa metodologia possibilita uma melhor organização das atividades, maior transparência do andamento do projeto e facilita a identificação de possíveis gargalos no processo de desenvolvimento. Além disso, o Kanban permite que a equipe acompanhe de forma simples o status de cada tarefa, garantindo maior controle e produtividade.

Outro benefício do uso do Kanban é a flexibilidade na gestão das demandas, permitindo priorizar atividades de acordo com as necessidades do projeto, bem como realizar ajustes no planejamento durante o desenvolvimento do sistema.

Trello

4 Prototipagem (Figma)

A prototipagem do sistema PredialFix foi realizada com o uso do Figma, com o objetivo de representar visualmente as principais telas e funcionalidades antes do desenvolvimento.

Foram criadas interfaces como login, abertura de chamados e acompanhamento de status, priorizando uma navegação simples e intuitiva. Essa etapa permitiu validar a organização das informações e melhorar a experiência do usuário.

Além disso, a prototipagem contribuiu para reduzir erros no desenvolvimento e facilitar o entendimento do funcionamento do sistema, garantindo maior qualidade no resultado final.

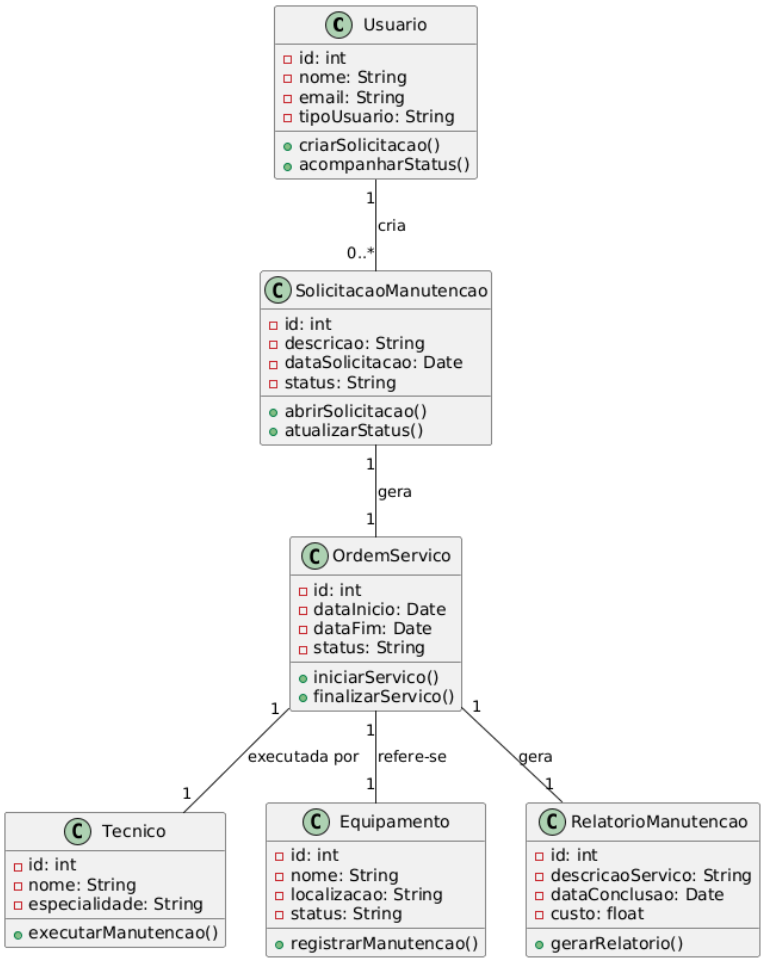
Figma

5 Diagrama de Classes e Banco de Dados

O Diagrama de Classes é um dos principais diagramas da UML, utilizado para representar a estrutura estática do sistema, incluindo classes, atributos, métodos e seus relacionamentos.

No sistema de Gestão de Manutenção Predial, ele modela elementos como usuários, ativos, ordens de serviço e técnicos. Cada classe representa uma entidade do sistema, permitindo visualizar como esses componentes se relacionam e interagem.
Esse diagrama contribui para a organização da arquitetura do sistema, facilitando o desenvolvimento e o entendimento por parte da equipe.

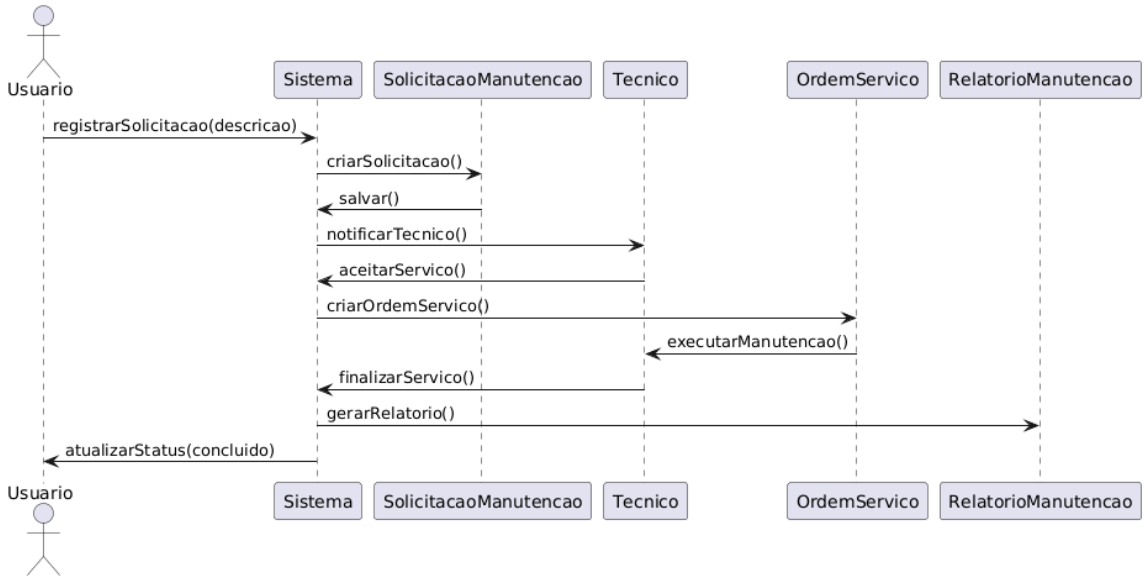
Além disso, a modelagem também serve como base para o banco de dados relacional, onde cada classe corresponde a uma tabela e seus atributos aos campos. Os relacionamentos são implementados por meio de chaves estrangeiras, garantindo a integridade dos dados e facilitando a integração com o Eloquent ORM no MySQL.



6 Diagrama de Sequência

O Diagrama de Sequência é um diagrama comportamental da UML utilizado para representar a **interação entre os elementos do sistema ao longo do tempo**. Esse tipo de diagrama demonstra como os objetos ou atores do sistema se comunicam por meio de mensagens para realizar determinada funcionalidade.

No sistema de Gestão de Manutenção Predial, o diagrama de sequência foi utilizado para representar o fluxo de interação entre o usuário, o sistema e os demais componentes durante a execução de processos importantes, como a abertura e o gerenciamento de uma ordem de serviço.

Nesse diagrama é possível observar a sequência de mensagens trocadas entre os participantes do sistema, desde o momento em que o usuário registra uma solicitação de manutenção até a criação da ordem de serviço, a execução do trabalho pelo técnico e o encerramento do serviço com a geração do relatório final.

A utilização do diagrama de sequência permite compreender de forma clara o fluxo de funcionamento do sistema, facilitando a análise dos processos, a identificação de possíveis melhorias e o alinhamento entre os requisitos do sistema e sua implementação.